

Дизайн и анализ на алгоритми упражнение 3

Станислав Игов (Стъката)

2025/2026

Рекурсивна версия на алгоритъма на Евклид. Доказателство на Рекурсивни алгоритми чрез индукция. Една задача с масив. Задача за максимална сума на подмасив (Maximum subarray sum)

1 Верификация на рекурсивни алгоритми: по индукция

Доказателство за коректност на рекурсивен алгоритъм Alg се прави по индукция по големината на входа. Базата на доказателството отговаря на спирачките на рекурсията - проверяваме, че за всеки вход x с големина, която задейства „спирачка“, $Alg(x)$ връща точно това, което очакваме. Следва същинската част на доказателството. Разглеждаме вход w с големина, която не задейства „спирачка“. Това значи, че $Alg(w)$ прави едно или повече извиквания върху входове с по-малки големина. Допускаме, че всяко от тези извиквания връща точно това, което очакваме - това е индуктивното предположение - и въз основа на допускането доказваме, че $Alg(w)$ връща точно това, което очакваме.

2 Рекурсивна версия на алгоритъма на Евклид

```
EuclidRec(a, b: positive integers)
@1  if a = 0 or b = 0 then
@2      return max(a, b)
@3  endif
@4  if a > b then
@5      return EuclidRec(a mod b, b)
@6  else
@7      return EuclidRec(a, b mod a)
@8  endif
```

3 Доказателство за коректност на алгоритъма на Евклид

Твърдение

За всички $a, b \in N$ функцията $EuclidRec(a, b)$ завършва и връща $\gcd(a, b)$.

Лема

Ще използваме известни свойства на НОД.

Лема 1. Ако $b > 0$, тогава

$$\gcd(a, b) = \gcd(a \bmod b, b).$$

Лема 2. Ако $a > 0$, тогава

$$\gcd(a, b) = \gcd(a, b \bmod a).$$

И двете следват от представянето

$$a = q \cdot b + (a \bmod b),$$

както и от факта, че множествата от общи делители на (a, b) и $(a \bmod b, b)$ съвпадат (аналогично за другата формула).

Терминация

Разглеждаме мярката

$$m(a, b) = a + b.$$

Ще покажем, че при всяко рекурсивно извикване тя строго намалява.

- Ако $a = 0$ или $b = 0$, алгоритъмът връща стойност и приключва.
- Ако $a > b > 0$, правим извикване $(a \bmod b, b)$. Тук $0 \leq a \bmod b < b$, следователно

$$(a \bmod b) + b < b + b \leq a + b.$$

- Ако $b \geq a > 0$, правим извикване $(a, b \bmod a)$. Тук $0 \leq b \bmod a < a$, следователно

$$a + (b \bmod a) < a + a \leq a + b.$$

Следователно $m(a, b)$ строго намалява и не може да намалява безкрайно в N . Значи рекурсията винаги достига случай с нула и алгоритъмът терминира.

Доказателство за коректност със силна индукция

Ще използваме силна индукция по наредени двойки (a, b) , подредени чрез мярката $a + b$ (еквивалентно: „по-малки“ са тези с по-малка сума).

База

Нека $a = 0$ или $b = 0$. Тогава алгоритъмът връща $\max(a, b)$.

По дефиниция:

$$\gcd(0, b) = b, \quad \gcd(a, 0) = a.$$

Следователно $\max(a, b) = \gcd(a, b)$. Базата е доказана.

Индуктивно предположение

Нека за дадена двойка (a, b) приемем, че за всяка двойка (p, q) с

$$p + q < a + b$$

алгоритъмът $\text{EuclidRec}(p, q)$ връща $\gcd(p, q)$.

Индуктивна стъпка

Разглеждаме $\text{EuclidRec}(a, b)$ при $a > 0$ и $b > 0$.

Случай 1: $a > b$. Алгоритъмът връща $\text{EuclidRec}(a \bmod b, b)$.

Понеже $(a \bmod b) + b < a + b$, по индуктивното предположение:

$$\text{EuclidRec}(a \bmod b, b) = \gcd(a \bmod b, b).$$

По Лема 1:

$$\gcd(a \bmod b, b) = \gcd(a, b).$$

Следователно върнатата стойност е $\gcd(a, b)$.

Случай 2: $a \leq b$. Алгоритъмът връща $\text{EuclidRec}(a, b \bmod a)$.

Понеже $a + (b \bmod a) < a + b$, по индуктивното предположение:

$$\text{EuclidRec}(a, b \bmod a) = \gcd(a, b \bmod a).$$

По Лема 2:

$$\gcd(a, b \bmod a) = \gcd(a, b).$$

И в този случай върнатата стойност е $\gcd(a, b)$.

Следователно индуктивната стъпка е доказана.

Заклучение

По силна индукция и доказаната терминация следва, че за всички $a, b \in \mathbb{N}$ алгоритъмът $\text{EuclidRec}(a, b)$ завършва и връща $\gcd(a, b)$.

4 Една задача с масив

Масив $A[1..n]$ от цели числа ще наричаме *Мартеница*, ако е изпълнено:

$$A[1] \leq A[2] \geq A[3] \leq A[4] \geq \dots$$

или формално: за $\forall k \in \{1, \dots, n-1\}$: $\begin{cases} A[k] \leq A[k+1], & \text{ако } k \text{ е нечетно,} \\ A[k] \geq A[k+1], & \text{ако } k \text{ е четно.} \end{cases}$

Да се предложи алгоритъм който решава следният проблем за време $O(n)$.

Вход: масив $A[1..n]$ от цели числа.

Изход: A' , модифицирана версия на A , така че A' е мартеница.

Примери:

- Вход: 1 2 3 4 5 6 7 8
Изход: 1 3 2 5 4 7 6 8
- Вход: 6 1 4 2 8 7 3 5
Изход: 1 6 2 8 4 7 3 5

Алгоритъм

```
Braid(A[1..n] : array of int)
@1 for i <- 1 to n-1 do
@2     if i mod 2 = 0 then
@3         if A[i] < A[i+1] then // искаме A[i] >= A[i+1] => разменяме
@4             swap(A[i], A[i+1])
@5         endif
@6     else
@7         if A[i] > A[i+1] then // искаме A[i] <= A[i+1] => разменяме
@8             swap(A[i], A[i+1])
@9         endif
@10    endif
@11 endfor
@12 return A
```

Нека A_i е състоянието на масива A преди i -тата итерация на цикъла на ред ③1. Дефинираме:

$$I(i) \iff A_i[1..i] \text{ е мартеница.}$$

Формално:

$$I(i) \iff \forall k \in \{1, \dots, i-1\} : \begin{cases} A_i[k] \leq A_i[k+1], & \text{ако } k \text{ е нечетно,} \\ A_i[k] \geq A_i[k+1], & \text{ако } k \text{ е четно.} \end{cases}$$

Доказателство

База

За $i = 1$ имаме $A_1[1..i] = A_1[1..1] = A_1[1]$. Множеството $\{1, \dots, 0\}$ е празно, следователно условието във формалната дефиниция е изпълнено тривиално. Значи $I(1)$ е вярно.

Поддръжка

Да допуснем, че инвариантът е верен преди итерация i , т.е. $A_i[1..i]$ е мартеница. По време на i -тата итерация проверяваме двойката $(i, i+1)$.

Случай 1: i е четно ($i \bmod 2 = 0$). Тогава искаме $A[i] \geq A[i+1]$.

- Ако $A_i[i] < A_i[i+1]$, изпълняваме ④ и разменяме:

$$A_{i+1}[i] = A_i[i+1] > A_i[i] = A_{i+1}[i+1],$$

тоест $A_{i+1}[i] > A_{i+1}[i+1]$.

Забелязваме, че се разменят само елементите на позиции i и $i+1$, следователно

$$A_i[1..i-1] = A_{i+1}[1..i-1].$$

От индуктивното допускане и понеже i е четно, имаме

$$A_i[i-1] \leq A_i[i].$$

Понеже $A_i[i-1]$ не се променя, $A_i[i-1] = A_{i+1}[i-1]$. А от

$$A_i[i-1] \leq A_i[i] < A_i[i+1]$$

след размяната следва

$$A_{i+1}[i-1] < A_{i+1}[i] \quad \text{и} \quad A_{i+1}[i] > A_{i+1}[i+1],$$

което гарантира, че условията за мартеница се запазват за префикса $[1..i+1]$.

- Ако $A_i[i] \geq A_i[i+1]$, не разменяме и директно имаме

$$A_{i+1}[i] = A_i[i] \geq A_i[i+1] = A_{i+1}[i+1],$$

следователно $A_{i+1}[1..i+1]$ е мартеница.

Случай 2: i е нечетно. Тогава искаме $A[i] \leq A[i+1]$ и аналогично: ако $A_i[i] > A_i[i+1]$ разменяме, иначе оставяме както е; във всички случаи условието за мартеница върху двойката $(i, i+1)$ се удовлетворява, без да се нарушава вече коректният префикс $A_i[1..i]$. Следователно $I(i+1)$ е вярно.

Така инвариантът се поддържа.

Терминация

Цикълът на ред ③1 изпълнява краен брой итерации: за $i = 1, 2, \dots, n-1$. Когато проверката на цикъла стане лъжа, имаме $i = n$ (т.е. цикълът е приключил).

От инварианта $I(n)$ следва, че $A_n[1..n]$ е мартеница, т.е.

$$\forall k \in \{1, \dots, n-1\} : \begin{cases} A_n[k] \leq A_n[k+1], & \text{ако } k \text{ е нечетно,} \\ A_n[k] \geq A_n[k+1], & \text{ако } k \text{ е четно.} \end{cases}$$

Следователно върнатият масив е мартеница.

Сложност

Алгоритъмът прави една линейна обиколка на масива и извършва константен брой операции на итерация, следователно времевата сложност е $O(n)$.

5 Алгоритъм на Кадан с префиксни суми

Да се предложи алгоритъм който решава следният проблем за време $O(n)$.

Вход: масив $[1..n]$ от цели числа.

Изход: m , където е максималната сума на подмасив на $A[1..n]$ или формално $m = \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j A[k]$

Примери:

$[-2, 7, -3, 4] \Rightarrow 8$ най-голямата сума е на подмасива $[7, -3, 4]$

$[-1, -2, -3, -4] \Rightarrow 0$ най-голямата сума е на празният подмасив $[]$

Идея: Префиксни суми

Префиксните суми са техника, която позволява бързо изчисляване на сумата на произволен подинтервал от масив.

Нека е даден масив $A[1..n]$.

Дефинираме масива от префиксни суми $P[0..n]$:

$$P[0] = 0$$

$$P[i] = A[1] + A[2] + \dots + A[i] = \sum_{t=1}^i A[t]$$

Тоест $P[i]$ съдържа сумата на първите i елемента.

Основно свойство на префиксните суми

Сумата на произволен подмасив $A[l..r]$ може да се изчисли чрез:

$$\text{sum}(l, r) = A[l] + A[l+1] + \dots + A[r] = P[r] - P[l-1]$$

Това свойство позволява да преобразуваме задачи за сума на интервал в операции върху префиксни суми

Важна последица (често използвана)

Ако искаме да максимизираме сумата на подмасив, завършващ в позиция r , имаме:

$$\max_{1 \leq l \leq r} \text{sum}(l, r) = \max_{1 \leq l \leq r} (P[r] - P[l-1])$$

Тъй като $P[r]$ е фиксирано, максимумът се получава когато $P[l-1]$ е минимален.

Следователно:

$$\max_{1 \leq l \leq r} \text{sum}(l, r) = P[r] - \min_{0 \leq k \leq r-1} P[k]$$

Интуиция

За всеки край на подмасив r :

- знаем текущата префиксна сума $P[r]$
- за да получим най-голямата сума, трябва да извадим най-малката префиксна сума преди него

Тоест:

Най-добрият подмасив, който завършва в r , започва след най-малкия префикс преди r .

Алгоритъм

```
KadanePrefixSum(A[1..n]: array of int)
@1 pref <- 0
@2 minPref <- 0
@3 best <- 0
@4 for i <- 1 to n do
@5   pref <- pref + A[i]
@6   best <- max(best, pref - minPref)
@7   minPref <- min(minPref, pref)
@8 endfor
@9 return best
```

$pref$ ще е текущата префиксна сума $P[i]$
 $minPref$ ще е минимума на префиксите досега
 $best$ ще е най-доброто намерено до момента.

Нека $pref_i, minPref_i, best_i$ са състоянията съответно на $pref, minPref$ и $best$ преди i -тата итерация на цикъла на ред @4

$$I(i) \Leftrightarrow \begin{cases} pref_i = P[i-1] \\ best_i = \max(0, \max_{1 \leq r \leq i-1} (P[r] - \min_{0 \leq k \leq r-1} P[k])) \\ minPref_i = \min P[0..i-1] \end{cases}$$

Доказателство

База: за $i = 1$

$$\begin{aligned} pref_1 &= 0 = P[1-1] = P[0] = 0 \\ minPref_1 &= 0 = \min P[0..1-1] = \min P[0..0] = P[0] = 0 \\ best_1 &= 0 = \max(0, \max_{1 \leq r \leq 1-1=0} (P[r] - \min_{0 \leq k \leq r-1} P[k])) = \max(0, \max(\emptyset)) = \max(0, -\infty) = 0 \end{aligned}$$

Поддръжка: Нека $I(i)$ е вярно за някое $i < n$ Разглеждаме изпълнението на цикъла:

$$\text{Ред @5} \Rightarrow pref_{i+1} = pref_i + A[i] \text{ От } I(i) \Rightarrow pref_i = P[i-1] = \sum_{k=1}^{i-1} A[k] \text{ Понеже } pref_{i+1} = pref_i + A[i] = \sum_{k=1}^{i-1} A[k] + A[i] = \sum_{k=1}^i A[k]$$

$$\begin{aligned} \text{Ред @6} \Rightarrow best_{i+1} &= \max(best_i, Pref_{i+1} - minPref_i) = \\ \text{от допускането че } I(i) \text{ е вярно} & \\ &= \max(\max(0, \max_{1 \leq r \leq i-1} (P[r] - \min_{0 \leq k \leq r-1} P[k])), Pref_{i+1} - minPref_i) = \\ &= \max(\max(0, \max_{1 \leq r \leq i-1} (P[r] - \min_{0 \leq k \leq r-1} P[k])), P[i] - \min P[0..i-1]) \end{aligned}$$

$$\text{може да запишем } (P[i] - \min P[0..i-1]) \text{ като } \max_{i \leq r \leq i} (P[r] - \min_{0 \leq k \leq r-1} P[k])$$

следователно можем да запишем уравнението като:

$$\max(0, \max_{1 \leq r \leq i-1} (P[r] - \min_{0 \leq k \leq r-1} P[k]), \max_{i \leq r \leq i} (P[r] - \min_{0 \leq k \leq r-1} P[k]))$$

тук махнахме единият $\max$

Сега можем да обединим 2-рата и 3-тата стойност в едно уравнение и получаваме:

$$best_{i+1} = \max(0, \max_{1 \leq r \leq i} (P[r] - \min_{0 \leq k \leq r-1} P[k]))$$

На ред @7 $\minPref_{i+1} = \min(\minPref_i, pref_{i+1}) = \min(\minPref_i, P[i]) = \min(\min P[0..i-1], P[i]) = \min(P[0..i])$

С това Поддръжката е доказана

Терминация

Нека разгледаме алгоритъма когато условието за изпълнение на цикъла на ред @4 за последен път.

Тогава $i = n + 1$

Тогава за $I(n)$ е изпълнено че

$pref_{n+1} = P[n]$

$\minPref_{n+1} = \min P[0..n]$

$best_{n+1} = \max(0, \max_{1 \leq r \leq n} (P[r] - \min_{0 \leq k \leq r-1} P[k]))$

На ред @9 връщаме $best_{n+1}$.

По-рано видяхме че $\max_{1 \leq i \leq j \leq n} \sum_{k=1}^j A[k] = \max_{1 \leq i \leq j \leq n} sum(i, j) = \max_{1 \leq i \leq j \leq n} (P[j] - P[i - 1]) = \max_{1 \leq j \leq n} (P[j] - \min_{1 \leq i \leq j} P[i - 1])$ което е точно това което нашият алгоритъм връща

Времева сложност

@1, @2, @3 се изпълняват по 1 път като присвояването е константна операция @4 се изпълнява $n+1$ пъти като операциите са с константна сложност @5, @6, @7 се изпълняват по n пъти $\Rightarrow O(n)$

@9 $\Rightarrow O(1)$

Получаваме $1 + 1 + 1 + (n + 1) + n + n + n + 1 = 4n + 5 \in O(n)$